Estimating and applying detectability offsets

Analysing Ecological Data with Detection Error

Péter Sólymos

2026-04-23

Table of contents

Preamble **1**
Duration and distance intervals . . . 6
Different models . . . 13
Poisson GLM . . . 14
Negative Binomial GLM . . . 14
Zero inflated Poisson . . . 14
Occupancy . . . 14
Poisson GAM . . . 15
Poisson-Lognormal GLMM in lme4 . . . 15
Poisson-Lognormal GLMM in INLA . . . 15
Regression trees . . . 15
Compare results . . . 19
Propagating error . . . 21
Model validation . . . 23

Preamble

Note: to install INLA, you have to use:

```
install.packages("INLA",
    repos = "https://inla.r-inla-download.org/R/stable", dep=TRUE)
```

```r
library(mefa4) # data manipulation
```

```
Loading required package: Matrix
```

```
mefa4 0.3-12     2025-07-12
```

```r
library(mgcv) # GAMs
```

```
Loading required package: nlme

This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.
```

```r
library(pscl) # zero-inflated models
```

```
Classes and Methods for R originally developed in the
Political Science Computational Laboratory
Department of Political Science
Stanford University (2002-2015),
by and under the direction of Simon Jackman.
hurdle and zeroinfl functions by Achim Zeileis.
```

```r
library(lme4) # GLMMs
```

```
Attaching package: 'lme4'

The following object is masked from 'package:nlme':

    lmList
```

```r
library(MASS) # Negative Binomial GLM
library(partykit) # regression trees
```

```
Loading required package: grid

Loading required package: libcoin

Loading required package: mvtnorm
```

```r
library(intrval) # interval magic
```

```
Attaching package: 'intrval'

The following object is masked from 'package:mefa4':

    %notin%
```

```r
library(opticut) # optimal partitioning
```

```
Loading required package: pbapply

opticut 0.1-4    2025-07-12
```

```r
library(visreg) # regression visualization
library(ResourceSelection) # marginal effects
```

```
ResourceSelection 0.3-6      2023-06-27
```

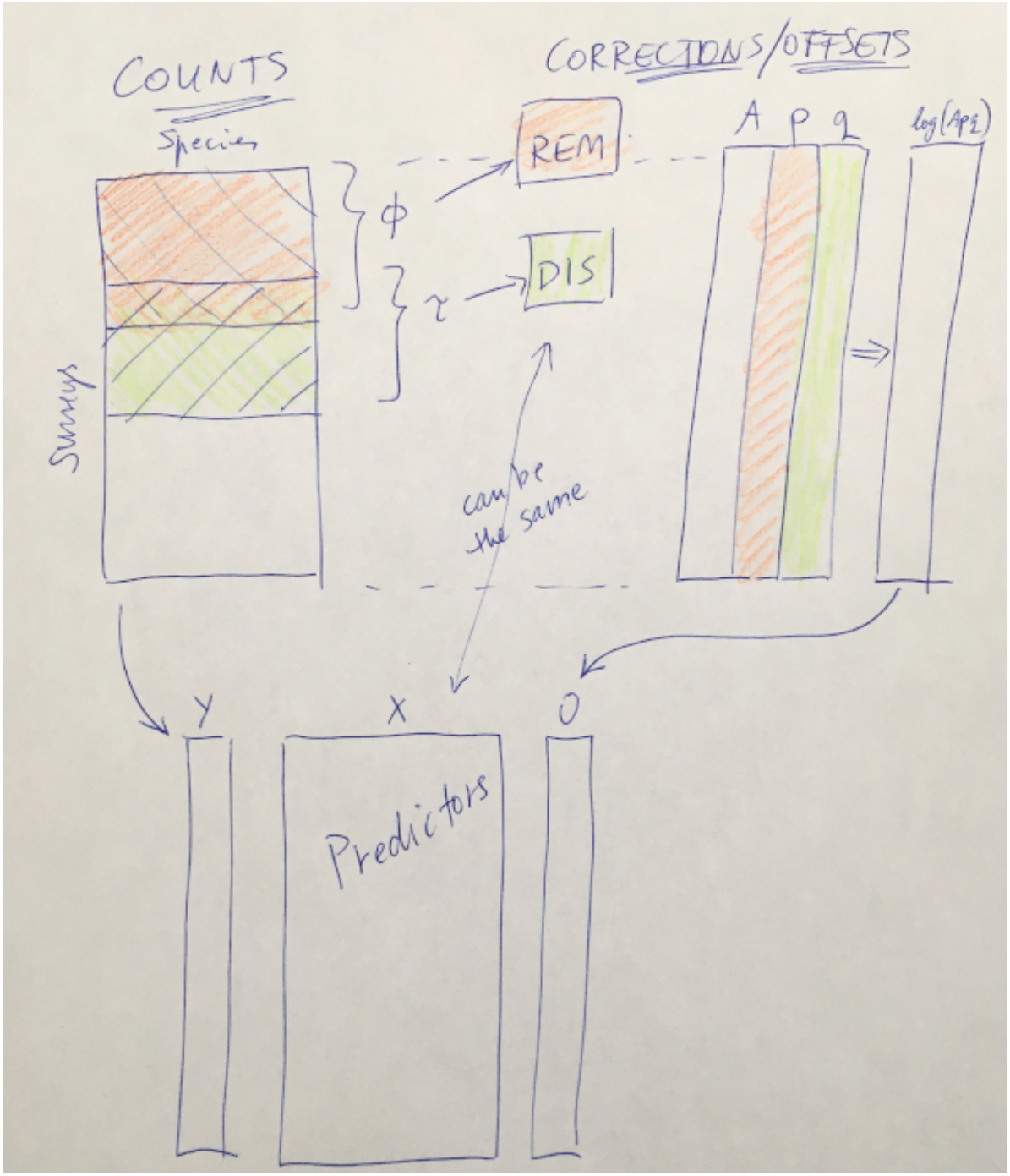
```r
library(MuMIn) # multi-model inference
library(INLA) # Integrated Nested Laplace Approximation
```

```
This is INLA_25.10.19 built 2025-10-19 19:16:33 UTC.
 - See www.r-inla.org/contact-us for how to get help.
 - List available models/likelihoods/etc with inla.list.models()
 - Use inla.doc(<NAME>) to access documentation
```

```r
library(detect)
```

```
Loading required package: Formula

Loading required package: stats4

detect 0.5-1     2026-01-04
```

```r
set.seed(1234)

x <- detect::josm$surveys
x$FOR <- x$Decid + x$Conif + x$ConifWet # forest
x$AHF <- x$Agr + x$UrbInd + x$Roads # 'alienating' human footprint
x$WET <- x$OpenWet + x$ConifWet + x$Water # wet + water
cn <- c(
    "Open", "Water", "Agr", "UrbInd", "SoftLin", "Roads", "Decid",
    "OpenWet", "Conif", "ConifWet"
)
x$HAB <- droplevels(find_max(x[, cn])$index) # drop empty levels
levels(x$HAB)[levels(x$HAB) %in%
    c("OpenWet", "Water", "Open", "Agr", "UrbInd", "Roads")] <- "Open"
levels(x$HAB)[levels(x$HAB) %in%
    c("Conif", "ConifWet")] <- "Conif"
x$OBS <- as.factor(x$ObserverID)
```


COUNTS
CORRECTIONS/OFFSETS
Species
Surveys
REM
DIS
A
p
q
log(Apq)
can be the same
Y
X
O
Predictors

Duration and distance intervals

```
cts <- josm$counts[josm$counts$DetectType1 != "V", ]
cts$DurDis <- paste(cts$Dur, cts$Dis)
table(cts$DurDis)
```

```
   0-3min 0-50m    0-3min 100+m  0-3min 50-100m    3-5min 0-50m    3-5min 100+m
          15520            3424           13981            2627            1265
 3-5min 50-100m   5-10min 0-50m   5-10min 100+m 5-10min 50-100m
           3130            4285            1893            4863
```

```
ydurdis <- Xtab(~ SiteID + DurDis + SpeciesID, cts)
```

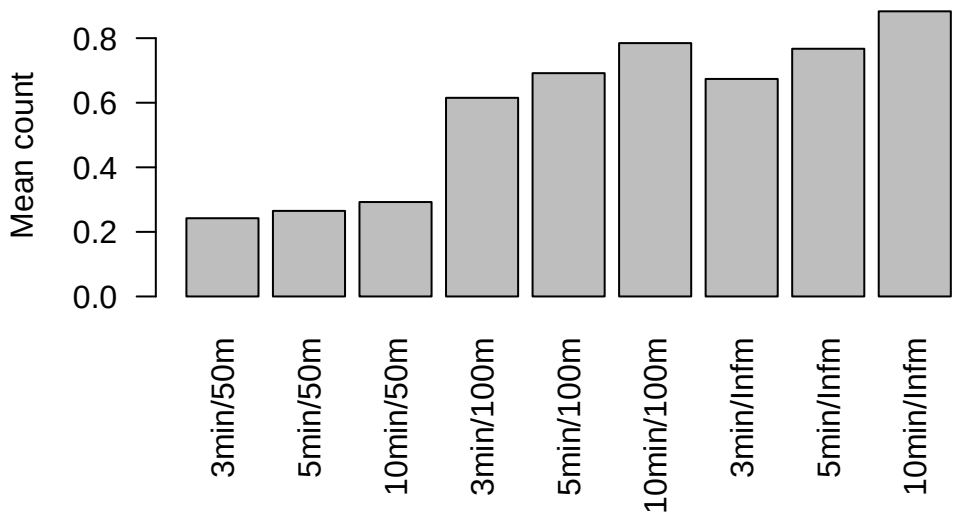
Create count for different methodologies

```
spp <- "OVEN"
y <- as.matrix(ydurdis[[spp]])
yc <- cbind(
    "3min/50m" = y[, "0-3min 0-50m"],
    "5min/50m" = y[, "0-3min 0-50m"] + y[, "3-5min 0-50m"],
    "10min/50m" = rowSums(y[, c("0-3min 0-50m", "3-5min 0-50m", "5-10min 0-50m")]),
    "3min/100m" = rowSums(y[, c("0-3min 0-50m", "0-3min 50-100m")]),
    "5min/100m" = rowSums(y[, c(
        "0-3min 0-50m", "3-5min 0-50m",
        "0-3min 50-100m", "3-5min 50-100m"
    )]),
    "10min/100m" = rowSums(y[, c(
        "0-3min 0-50m", "3-5min 0-50m", "5-10min 0-50m",
        "0-3min 50-100m", "3-5min 50-100m", "5-10min 50-100m"
    )]),
    "3min/Infm" = rowSums(y[, c(
        "0-3min 0-50m",
        "0-3min 50-100m", "0-3min 100+m"
    )]),
    "5min/Infm" = rowSums(y[, c(
        "0-3min 0-50m", "3-5min 0-50m",
        "0-3min 50-100m", "3-5min 50-100m",
        "0-3min 100+m", "3-5min 100+m"
    )]),
```

```
    "10min/Infm" = rowSums(y[, c(
        "0-3min 0-50m", "3-5min 0-50m", "5-10min 0-50m",
        "0-3min 50-100m", "3-5min 50-100m", "5-10min 50-100m",
        "0-3min 100+m", "3-5min 100+m", "5-10min 100+m"
    )])
)
```

See how mean counts are different:

```
op <- par(mar = c(8, 4, 2, 2), las = 2)
barplot(colMeans(yc), ylab = "Mean count")
```




```
par(op)
```

Removal model

```
Ydur <- as.matrix(Xtab(~ SiteID + Dur + SpeciesID, cts)[[spp]])
Ddur <- matrix(c(3, 5, 10), nrow(Ydur), 3, byrow = TRUE)
Mdur <- cmulti(Ydur | Ddur ~ DAY, x[rownames(Ydur), ], type = "rem")

phi <- drop(exp(model.matrix(Mdur) %*% coef(Mdur)))
summary(phi)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.4086  0.4196  0.4278  0.4290  0.4394  0.4512
```

Distance sampling

```
Ydis <- as.matrix(Xtab(~ SiteID + Dis + SpeciesID, cts)[[spp]])
Ddis <- matrix(c(0.5, 1, Inf), nrow(Ydis), 3, byrow = TRUE)
Mdis <- cmulti(Ydis | Ddis ~ HAB, x[rownames(Ydur), ], type = "dis")

tau <- drop(exp(model.matrix(Mdis) %*% coef(Mdis)))
summary(tau)
```

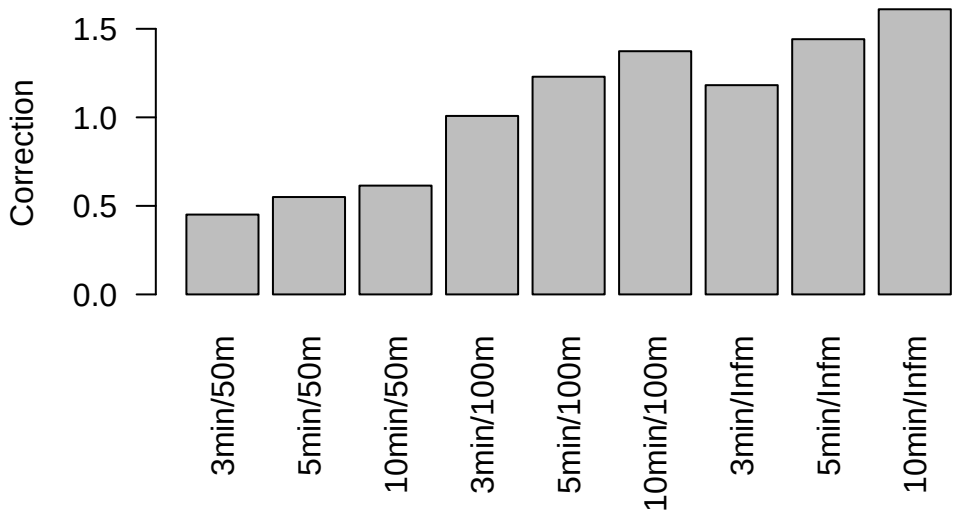
```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
 0.6810  0.7000  0.7000  0.7206  0.7429  0.7429 
```

Calculating offsets for the different methodologies (max duration and max distance)

```
p_fun <- function(t, phi) {
    1 - exp(-t * phi)
}
q_fun <- function(r, tau) {
    tau^2 * (1 - exp(-r^2 / tau^2)) / r^2
}

## availability
p3 <- p_fun(3, phi)
p5 <- p_fun(5, phi)
p10 <- p_fun(10, phi)

## average detection
q50 <- q_fun(0.5, tau)
q100 <- q_fun(1, tau)
## area sampled (known)
A50 <- 0.5^2 * pi
A100 <- 1^2 * pi
## effective area sampled
EA <- tau^2 * pi

## correction factors: C=Apq
Corr <- cbind(
    "3min/50m" = A50 * p3 * q50,
    "5min/50m" = A50 * p5 * q50,
    "10min/50m" = A50 * p10 * q50,
    "3min/100m" = A100 * p3 * q100,
    "5min/100m" = A100 * p5 * q100,
```

```
    "10min/100m" = A100 * p10 * q100,
    "3min/Infm" = EA * p3 * 1,
    "5min/Infm" = EA * p5 * 1,
    "10min/Infm" = EA * p10 * 1
)
```

```
op <- par(mar = c(8, 4, 2, 2), las = 2)
barplot(colMeans(Corr), ylab = "Correction")
```



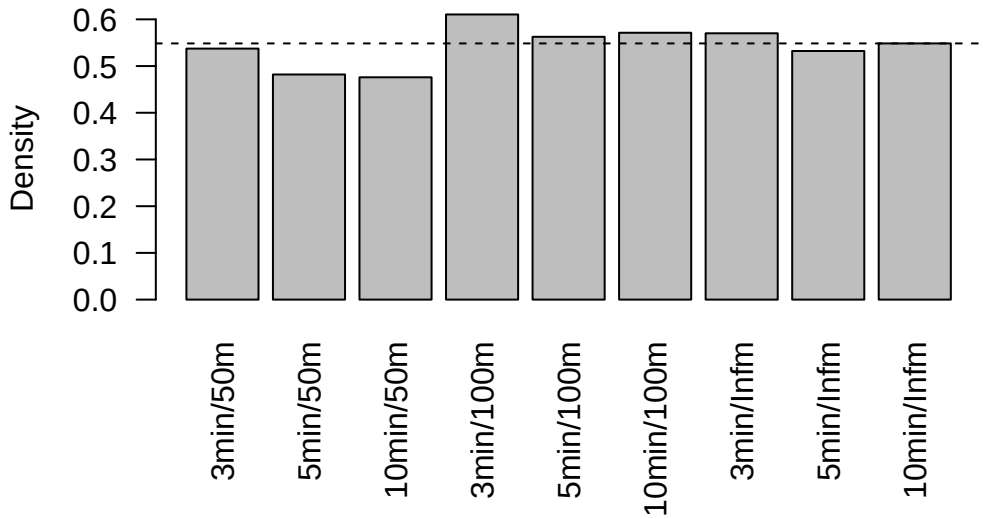

```
par(op)
```

See how $E[Y] = DApq$ and thus $\hat{D} = E[Y]/Apq = E[Y]/C$ is going to work. Taking the mean of the Apq term that varies from row to row is not strictly correct (it is correct for constant φ and τ)

```
(D0 <- colMeans(yc) / colMeans(Corr))
```

```
  3min/50m   5min/50m  10min/50m  3min/100m  5min/100m 10min/100m  3min/Infm
 0.5373668  0.4819564  0.4759077  0.6104367  0.5626037  0.5712841  0.5701134
 5min/Infm 10min/Infm
 0.5322467  0.5484187
```

```
op <- par(mar = c(8, 4, 2, 2), las = 2)
barplot(D0, ylab = "Density")
abline(h = D0["10min/Infm"], lty = 2)
```



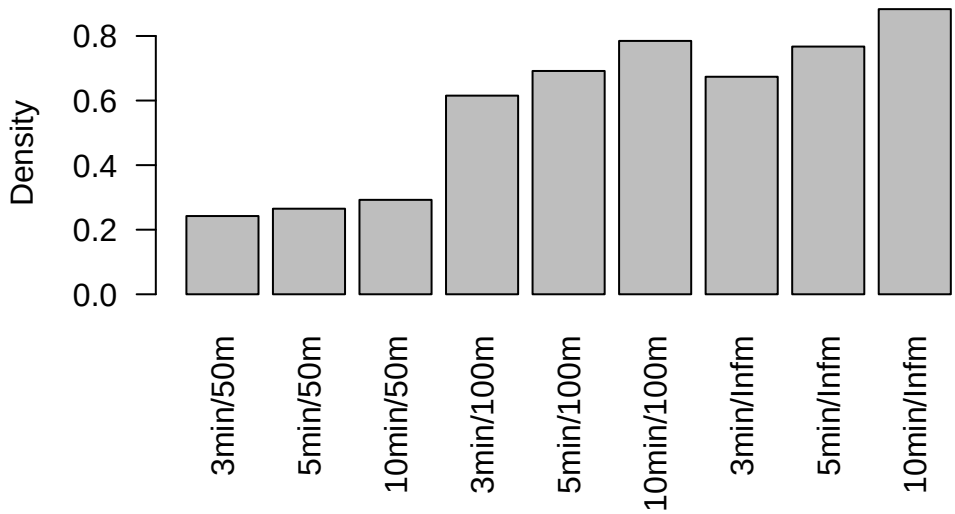

```
par(op)
```

Naive GLM

```
M1 <- apply(yc, 2, function(z) {
    glm(z ~ Decid, data = x, family = poisson)
})
t(sapply(M1, coef))
```

```
           (Intercept)    Decid
3min/50m     -2.409761 2.051577
5min/50m     -2.338716 2.083917
10min/50m    -2.265081 2.126118
3min/100m    -1.566005 2.202281
5min/100m    -1.459256 2.219651
10min/100m   -1.326642 2.208799
3min/Infm    -1.442896 2.147316
5min/Infm    -1.322982 2.164338
10min/Infm   -1.164268 2.133832
```

```
DN <- sapply(M1, function(z) mean(fitted(z)))
op <- par(mar = c(8, 4, 2, 2), las = 2)
barplot(DN, ylab = "Density")
```




```
par(op)
```

GLM with offsets: offset is the log of the correction factor: $o_i = log(C_i) = log(A_i p_i, q_i)$

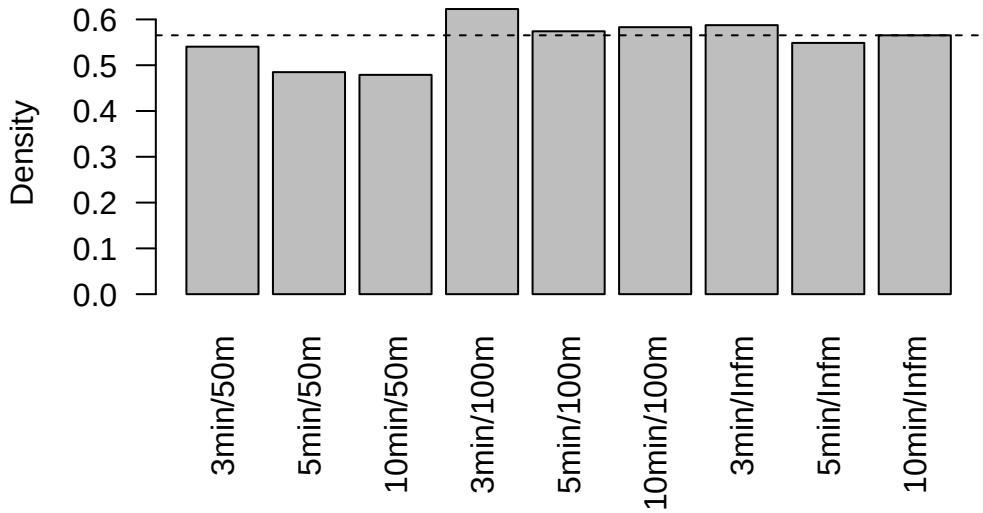
```
Off <- log(Corr)

M2 <- lapply(colnames(yc), function(i) {
    glm(yc[, i] ~ Decid, data = x, family = poisson, offset = Off[, i])
})
names(M2) <- colnames(yc)
t(sapply(M2, coef))
```

```
           (Intercept)    Decid
3min/50m     -1.623378 2.078982
5min/50m     -1.751248 2.112139
10min/50m    -1.788788 2.155212
3min/100m    -1.605653 2.289184
5min/100m    -1.697799 2.307331
10min/100m   -1.676477 2.297545
3min/Infm    -1.657984 2.279331
5min/Infm    -1.736939 2.297082
10min/Infm   -1.689671 2.267898
```

```
X <- model.matrix(M2[[1]])
DO <- sapply(M2, function(z) mean(exp(X %*% coef(z))))
DO
```

```
  3min/50m   5min/50m  10min/50m  3min/100m  5min/100m 10min/100m  3min/Infm
 0.5404376  0.4848354  0.4788956  0.6226936  0.5740894  0.5830293  0.5874727
 5min/Infm 10min/Infm
 0.5486705  0.5652759
```

```
op <- par(mar = c(8, 4, 2, 2), las = 2)
barplot(DO, ylab = "Density")
abline(h = DO["10min/Infm"], lty = 2)
```




```
par(op)
```

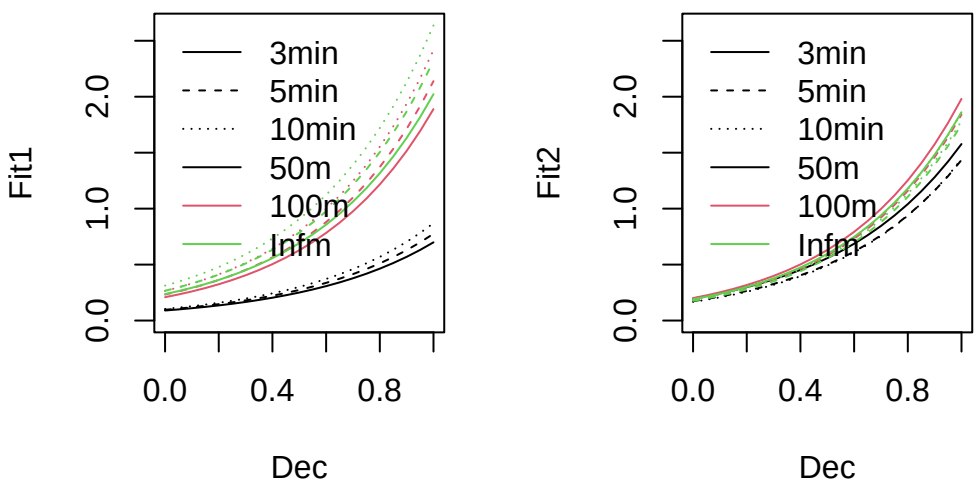
Compare the naive and the offsetted GLM

```
Dec <- seq(0, 1, 0.05)
Xpred <- cbind(1, Dec)
Fit1 <- sapply(M1, function(z) drop(exp(Xpred %*% coef(z))))
Fit2 <- sapply(M2, function(z) drop(exp(Xpred %*% coef(z))))
```

The 50 m counts under, the 100 and unlimited counts overestimated density compared to the estimate using the offsets

```
op <- par(mfrow = c(1, 2))
matplot(Dec, Fit1,
    type = "l", ylim = c(0, max(Fit1, Fit2)),
    col = rep(1:3, each = 3), lty = rep(1:3, 3)
)
```

```r
legend("topleft",
    bty = "n", col = c(1, 1, 1, 1, 2, 3), lty = c(1, 2, 3, 1, 1, 1),
    legend = c("3min", "5min", "10min", "50m", "100m", "Infm")
)
matplot(Dec, Fit2,
    type = "l", ylim = c(0, max(Fit1, Fit2)),
    col = rep(1:3, each = 3), lty = rep(1:3, 3)
)
legend("topleft",
    bty = "n", col = c(1, 1, 1, 1, 2, 3), lty = c(1, 2, 3, 1, 1, 1),
    legend = c("3min", "5min", "10min", "50m", "100m", "Infm")
)
```




```r
par(op)
```

Different models

Let's explore models and develop:

- estimation with offset
- prediction without offset

This will be a recipe book that you can use. We just did the Poisson GLM, but repeat it here with the 5 min 100 m counts (you can change this of course). We'll keep counts, offsets, and the fixed effects the same

```r
x <- x[order(x$Decid), ] # order according to Decid
y <- yc[rownames(x), "5min/100m"]
o <- Off[rownames(x), "5min/100m"]
X <- model.matrix(~Decid, x)
```

Poisson GLM

```r
## model fit
MP <- glm(y ~ Decid, data = x, family = poisson, offset = o)

## predicting D
DP <- drop(exp(X %*% coef(MP)))
```

Negative Binomial GLM

```r
MNB <- glm.nb(y ~ Decid + offset(o), data = x)

DNB <- drop(exp(X %*% coef(MNB)))
```

Zero inflated Poisson

```r
MZIP <- zeroinfl(y ~ Decid | 1, x, dist = "poisson", offset = o)

DZIP <- drop((1 - plogis(coef(MZIP, "zero"))) * exp(X %*% coef(MZIP, "count")))
```

```r
MZINB <- zeroinfl(y ~ Decid | 1, x, dist = "negbin", offset = o)

DZINB <- drop((1 - plogis(coef(MZINB, "zero"))) * exp(X %*% coef(MZINB, "count")))
```

Occupancy

```r
y01 <- ifelse(y > 0, 1, 0)
MB <- glm(y01 ~ Decid, data = x, family = binomial("cloglog"), offset = o)

## note: we are using exp instead of cloglog inverse link
DB <- drop(exp(X %*% coef(MB)))
```

Poisson GAM

```r
MPG <- mgcv::gam(y ~ s(Decid), data = x, family = poisson, offset = o)

DPG <- predict(MPG, newdata = x, type = "response")
```

Poisson-Lognormal GLMM in lme4

```r
MPLN <- glmer(y ~ Decid + (1 | SiteID), data = x, family = poisson, offset = o)

DPLN <- drop(exp(X %*% fixef(MPLN)))
```

Poisson-Lognormal GLMM in INLA

See a tutorial here

```r
MPLNI <- inla(y ~ Decid + f(SiteID, model = "iid"),
    data = x, family = "poisson"
)

DPLNI <- drop(exp(X %*% MPLNI$summary.fixed[, "mean"]))
```

Regression trees

```r
library(gbm)
```

```
Loaded gbm 2.2.3
```

```
This version of gbm is no longer under development. Consider transitioning to gbm3, https://g
developers/gbm3
```

```r
library(dismo)
```

```
Loading required package: raster

Loading required package: sp


Attaching package: 'raster'

The following object is masked from 'package:MASS':

    select

The following object is masked from 'package:lme4':

    getData

The following object is masked from 'package:nlme':

    getData
```

```r
library(ggplot2)

xi <- data.frame(
    y = y,
    x[, c(
        "Open", "Water", "Agr", "UrbInd", "SoftLin", "Roads",
        "Decid", "OpenWet", "Conif", "ConifWet", "FOR", "AHF", "WET"
    )]
)
```

```r
# this does k-fold cross validation
brt <- gbm.step(xi,
    gbm.y = 1,
    gbm.x = 2:ncol(xi),
    offset = o,
```

```r
    family = "poisson",
    tree.complexity = 3,
    learning.rate = 0.01,
    bag.fraction = 0.5
)
DBRT <- predict.gbm(brt, xi, brt$n.trees, type = "response")
```

```r
brt <- gbm(y ~ . + offset(o),
    data = xi,
    n.trees = 1000,
    interaction.depth = 3,
    shrinkage = 0.001,
    bag.fraction = 0.5,
    distribution = "poisson",
    var.monotone = NULL,
    keep.data = FALSE,
    verbose = FALSE,
    n.cores = 1
)
DBRT <- predict.gbm(brt, xi, brt$n.trees, type = "response")
```

```
Warning in predict.gbm(brt, xi, brt$n.trees, type = "response"): predict.gbm
does not add the offset to the predicted values.
```

Variable importance

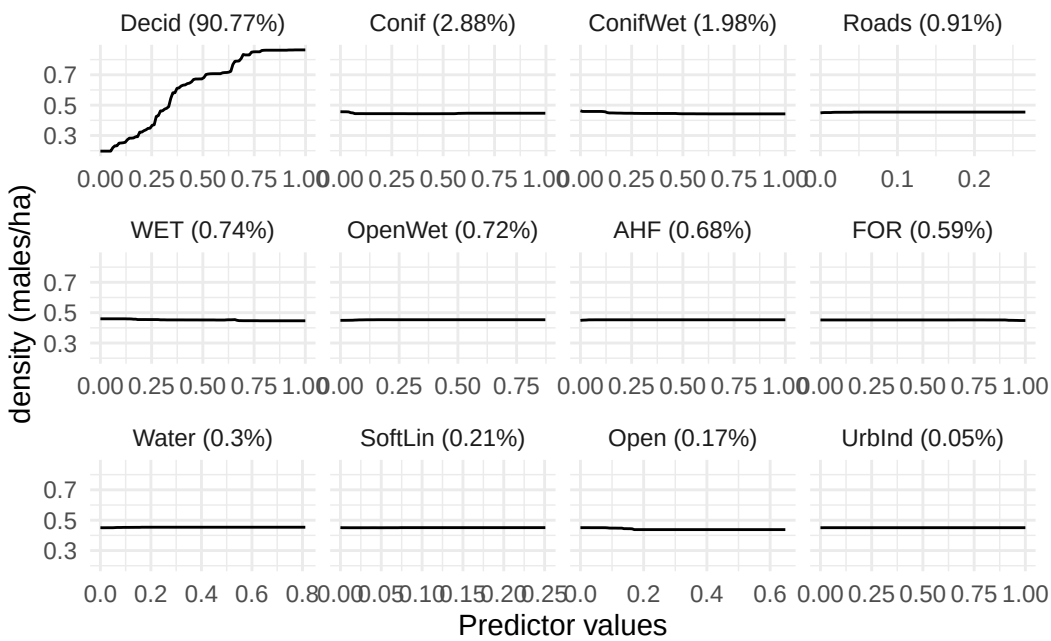
```r
rel_inf <- function(res) {
    rel.inf <- relative.influence(res, res$n.trees)
    rel.inf[rel.inf < 0] <- 0
    i <- order(-rel.inf)
    rel.inf <- 100 * rel.inf / sum(rel.inf)
    out <- data.frame(var = res$var.names[i], rel.inf = rel.inf[i])
    attr(out, "n.trees") <- res$n.trees
    out
}
rel_inf(brt)
```

```
              var     rel.inf
Decid       Decid 90.76983283
Conif       Conif  2.88255110
```

```
ConifWet ConifWet  1.97839768
Roads       Roads  0.90927936
WET           WET  0.74484388
OpenWet   OpenWet  0.71998223
AHF           AHF  0.67709984
FOR           FOR  0.59301105
Water       Water  0.29971632
SoftLin   SoftLin  0.20548144
Open         Open  0.16635701
UrbInd     UrbInd  0.05344726
Agr           Agr  0.00000000
```

Marginal effects plots

```r
.plot_fun <- function(i, res, u) {
    j <- as.character(u$var[i])
    x <- plot.gbm(res, j,
        n.trees = res$n.trees,
        return.grid = TRUE,
        type = "response",
        ylab = paste(res$rof_settings$spp, "density (males/ha)"),
        xlab = paste0(j, " (", round(u$rel.inf[i], 2), "%)")
    )
    colnames(x) <- c("x", "y")
    x$var <- paste0(j, " (", round(u$rel.inf[i], 2), "%)")
    attr(x, "out.attrs") <- NULL
    x
}
plot_fun <- function(res) {
    u <- rel_inf(res)
    xx <- do.call(rbind, lapply(1:12, .plot_fun, res, u))
    xx$var <- factor(xx$var, unique(xx$var))
    p <- ggplot(xx, aes(x = x, y = y)) +
        geom_line() +
        facet_wrap(vars(var), scales = "free_x") +
        ylab(paste(res$rof_settings$spp, "density (males/ha)")) +
        xlab("Predictor values") +
        theme_minimal()
    p
}
print(plot_fun(brt))
```



Compare results

```
AIC(MP, MNB, MZIP, MZINB, MPG, MPLN)
```

```
            df      AIC
MP    2.000000 9568.131
MNB   3.000000 9441.644
MZIP  3.000000 9471.389
MZINB 4.000000 9443.139
MPG   9.718238 9201.855
MPLN  3.000000 9402.942
```

```
mean(DP)
```

```
[1] 0.5740894
```

```
mean(DB)
```

```
[1] 0.5003848
```

```r
mean(DNB)
```

```
[1] 0.5829634
```

```r
mean(DZIP)
```

```
[1] 0.5641891
```

```r
mean(DZINB)
```

```
[1] 0.5807734
```

```r
mean(DPG)
```

```
[1] 0.5748426
```

```r
mean(DPLN)
```

```
[1] 0.4722719
```

```r
mean(DPLNI)
```

```
[1] 0.5960352
```

```r
mean(DBRT)
```

```
[1] 0.5250194
```

Propagating error

Use the Poisson GLM to show how to propagate uncertainty from the removal and distance model into the GLM

```
library(MASS)
B <- 100

Xdur <- model.matrix(Mdur)
(cfDur <- coef(Mdur))
```

```
log.phi_(Intercept)         log.phi_DAY
         -0.4496234          -0.8835723
```

```
(vcvDur <- vcov(Mdur))
```

```
                    log.phi_(Intercept) log.phi_DAY
log.phi_(Intercept)          0.08424602  -0.1879318
log.phi_DAY                 -0.18793177   0.4213039
```

```
cfBDur <- mvrnorm(B, cfDur, vcvDur)
head(cfBDur)
```

```
     log.phi_(Intercept) log.phi_DAY
[1,]           0.1834823  -2.2716186
[2,]          -0.5170667  -0.6974313
[3,]          -0.8452519   0.0143999
[4,]          -0.7273802  -0.3106298
[5,]          -0.5451327  -0.7447783
[6,]          -0.1966499  -1.5190658
```

```
phiB <- apply(cfBDur, 1, function(z) {
    drop(exp(Xdur %*% z))
})

p5B <- apply(phiB, 2, function(z) p_fun(5, z))
```

```
Xdis <- model.matrix(Mdis)
(cfDis <- coef(Mdis))
```

```
log.tau_(Intercept)    log.tau_HABDecid    log.tau_HABConif
        -0.38416880          0.02747205          0.08696738
```

```
(vcvDis <- vcov(Mdis))
```

```
                    log.tau_(Intercept) log.tau_HABDecid log.tau_HABConif
log.tau_(Intercept)         0.004111215     -0.004111215     -0.004111215
log.tau_HABDecid           -0.004111215      0.004216864      0.004111215
log.tau_HABConif           -0.004111215      0.004111215      0.004442186
```

```
cfBDis <- mvrnorm(B, cfDis, vcvDis)
tauB <- apply(cfBDis, 1, function(z) {
    drop(exp(Xdis %*% z))
})

q100B <- apply(tauB, 2, function(z) q_fun(1, z))

oB <- log(A100 * p5B * q100B)
```

```
COEF1 <- COEF2 <- matrix(0, B, 2)
for (i in 1:B) {
    ii <- sample.int(nrow(x), nrow(x), replace = TRUE)
    ## no error propagation
    m1 <- glm(y[ii] ~ Decid, data = x[ii, ], family = poisson, offset = o[ii])
    COEF1[i, ] <- coef(m1)
    ## error propagation
    m2 <- glm(y[ii] ~ Decid, data = x[ii, ], family = poisson, offset = oB[ii, i])
    COEF2[i, ] <- coef(m2)
}
apply(COEF1, 2, sd)
```

```
[1] 0.04402211 0.05597370
```

```
apply(COEF2, 2, sd)
```

```
[1] 0.04635428 0.05752563
```

```r
Xpred2 <- Xpred[c(1, 5, 10, 15, 20), ]
pr1 <- t(apply(
    exp(Xpred2 %*% t(COEF1)), 1,
    quantile, c(0.05, 0.95)
))
pr2 <- t(apply(
    exp(Xpred2 %*% t(COEF2)), 1,
    quantile, c(0.05, 0.95)
))
data.frame(none = pr1, prop = pr2)
```

```
    none.5.  none.95.   prop.5.  prop.95.
1 0.1705960 0.1964195 0.1777149 0.2065115
2 0.2751929 0.3057147 0.2788586 0.3147039
3 0.4976133 0.5365906 0.4891654 0.5377776
4 0.8928435 0.9528930 0.8501829 0.9256694
5 1.5876801 1.7131556 1.4703317 1.6260035
```

Model validation

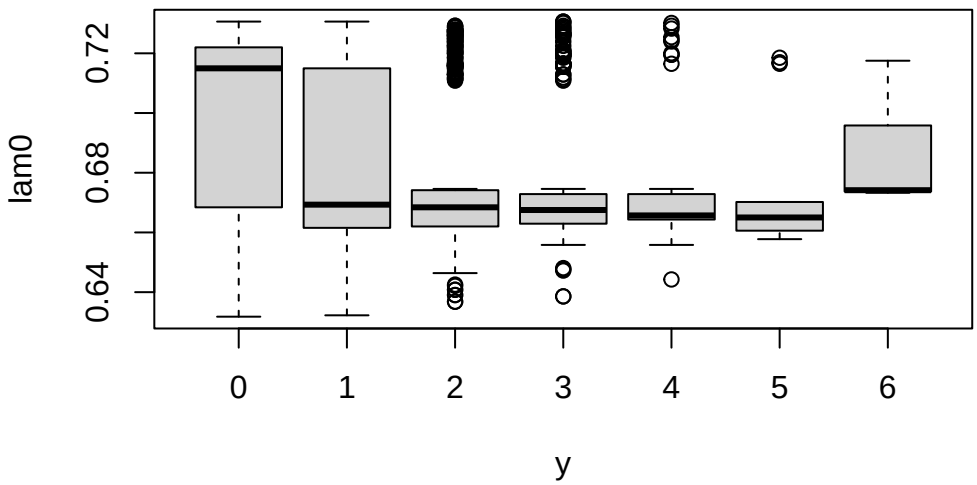
Using offsets can have some side effects. Let's inspect the performance of a null and fixed effect GLM.

When we look at observed vs. predicted based goodness of fit tests or AUC type metrics, we need $\lambda = DApq = De^{offset}$

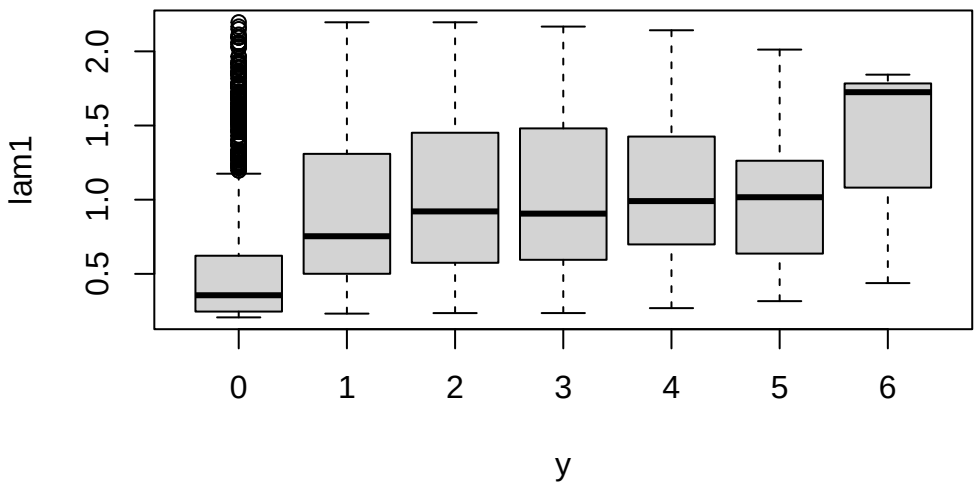
```r
M0 <- glm(y ~ 1, data = x, family = poisson, offset = o)
M1 <- glm(y ~ Decid, data = x, family = poisson, offset = o)

lam0 <- drop(exp(X[, 1, drop = FALSE] %*% coef(M0))) * exp(o)
lam1 <- drop(exp(X %*% coef(M1))) * exp(o)
```

Boxplot: when we have different methods mixed together the expected counts can vary greatly (not the case here, because we fixed methodology). Thus offsets can drive the goodness of fit metric.

```r
boxplot(lam0 ~ y)
```




```r
boxplot(lam1 ~ y)
```

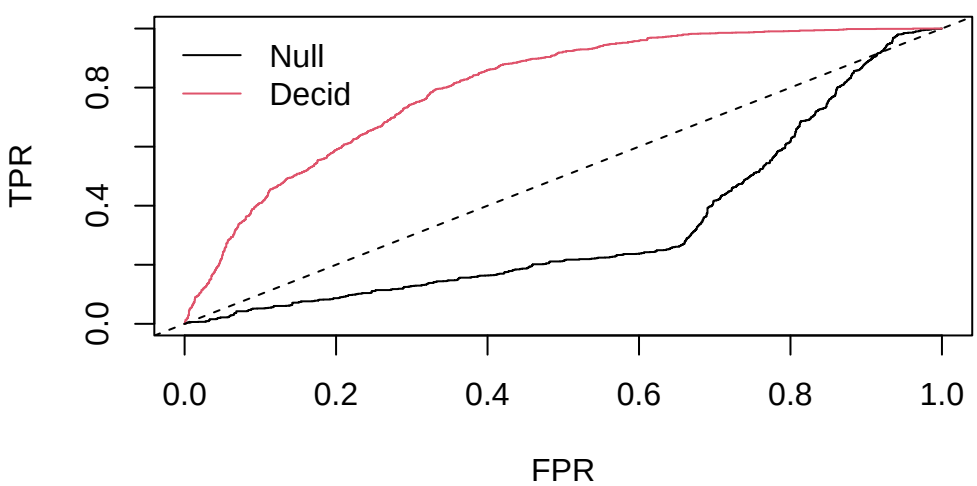



```r
simple_roc <- function(labels, scores) {
    Labels <- labels[order(scores, decreasing = TRUE)]
    data.frame(
        TPR = cumsum(Labels) / sum(Labels),
        FPR = cumsum(!Labels) / sum(!Labels),
        Labels = Labels
    )
}
simple_auc <- function(ROC) {
    ROC$inv_spec <- 1 - ROC$FPR
    dx <- diff(ROC$inv_spec)
    sum(dx * ROC$TPR[-1]) / sum(dx)
```

```r
}
roc0 <- simple_roc(ifelse(y > 0, 1, 0), lam0)
roc1 <- simple_roc(ifelse(y > 0, 1, 0), lam1)
(auc0 <- simple_auc(roc0))
```

```
[1] 0.328372
```

```r
(auc1 <- simple_auc(roc1))
```

```
[1] 0.798886
```

```r
plot(roc0[, 2:1], type = "l")
lines(roc1[, 2:1], col = 2)
abline(0, 1, lty = 2)
legend("topleft", bty = "n", lty = 1, col = c(1, 2), legend = c("Null", "Decid"))
```